

VÕ TIẾN

Thảo luận kiến thức CNTT trường BK về KHMT(CScience), KTMT(CEngineering)

<https://www.facebook.com/groups/khmt.ktmt.cse.bku>



Lập Trình Nâng Cao

LTNC - HK252

OOP cơ bản và 4 tính chất

Thảo luận kiến thức CNTT trường BK
về KHMT(CScience), KTMT(CEngineering)

<https://www.facebook.com/groups/khmt.ktmt.cse.bku>



1. [Class & Object - Basic] Trong Java, đối tượng được tạo từ?

```
1 class Student {  
2     String name;  
3 }  
4 Student s = new Student();
```

- a) Interface b) Constructor c) Class d) Method

2. [Constructor - Basic] Constructor là gì?

```
1 class Book {  
2     String title;  
3     Book(String t) {  
4         title = t;  
5     }  
6 }
```

- a) Phương thức dùng để khởi tạo đối tượng b) Một loại biến đặc biệt
c) Một lớp con d) Một interface

3. [this keyword - Basic] Từ khóa 'this' dùng để làm gì?

```
1 class Employee {  
2     String name;  
3     Employee(String name) {  
4         this.name = name;  
5     }  
6 }
```

- a) Truy cập biến toàn cục b) Gọi constructor mặc định
c) Trỏ đến đối tượng hiện tại d) Tạo object mới

4. [Object Array - Basic] Mảng các đối tượng dùng để làm gì?

```
1 Student[] students = new Student[10];
```

- a) Quản lý nhiều object cùng kiểu b) Tạo biến tĩnh
c) Gọi constructor d) Không dùng được trong Java

5. [final keyword - Basic] Từ khóa 'final' dùng với class nghĩa là?

```
1 final class Constants {  
2     public static final double PI = 3.14;  
3 }
```

- a) Class không thể kế thừa b) Class không thể tạo object
c) Class chỉ có static method d) Class không chứa biến

6. [Static method - Basic] Tại sao nên dùng phương thức static?

```
1 class MathUtil {  
2     public static int square(int x) {  
3         return x * x;  
4     }  
5 }
```



- a) Dễ gọi mà không cần tạo object
- b) Để tạo constructor riêng
- c) Để truy cập biến private
- d) Để overload constructor

7. [Nested Class - Basic] Lớp lồng nhau (nested class) được dùng khi nào?

```
1 class Outer {  
2     class Inner {  
3         void show() {  
4             System.out.println("Inside inner class");  
5         }  
6     }  
7 }
```

- a) Khi class chỉ được dùng trong một class khác
- b) Khi cần tạo static method
- c) Khi không có constructor
- d) Khi không muốn dùng package

8. [Garbage Collection - Basic] Java xử lý bộ nhớ ra sao?

- a) Sử dụng garbage collector tự động
- b) Người dùng phải giải phóng
- c) Dùng free() giống C
- d) Dùng delete keyword

9. [Reference vs Value - Basic] Điều nào đúng về truyền tham số?

```
1 void change(int x) { x = 10; }
```

- a) Java truyền tham trị với kiểu nguyên thủy
- b) Java luôn truyền tham chiếu
- c) Java không truyền tham số
- d) Java truyền con trỏ

10. [Object Comparison - Basic] So sánh object trong Java nên dùng gì?

```
1 String a = new String("hello");  
2 String b = new String("hello");  
3 System.out.println(a.equals(b)); // true
```

- a) equals()
- b) ==
- c) compareTo()
- d) hashCode()

11. [Static Block - Basic] Kết quả khi chạy đoạn mã sau là gì?

```
1 class InitDemo {  
2     static {  
3         System.out.println("Static block");  
4     }  
5     public static void main(String[] args) {  
6         System.out.println("Main method");  
7     }  
8 }
```

- a) Static block
- b) Main method
- c) Chỉ in Main method
- d) Lỗi biên dịch

12. [Reference Sharing - Basic] Kết quả đoạn code sau là?

```
1 class Data {  
2     int x;  
3 }  
4 public class Test {
```



```
5 public static void main(String[] args) {  
6     Data a = new Data();  
7     Data b = a;  
8     b.x = 5;  
9     System.out.println(a.x);  
10 }  
11 }
```

- a) 0
b) 5
c) Lỗi runtime
d) null

13. [Anonymous Object - Basic] Đoạn mã sau có ý nghĩa gì?

```
1 new Thread(new Runnable() {  
2     public void run() {  
3         System.out.println("Running");  
4     }  
5 }).start();
```

- a) Tạo và chạy luồng dùng object vô danh
b) Lỗi do thiếu tên class
c) Chỉ tạo object không chạy
d) Không chạy được trong Java

14. [Static vs Instance - Basic] Điều nào đúng khi gọi 'method()' dưới đây?

```
1 class Demo {  
2     static void method() {  
3         System.out.println("Static");  
4     }  
5 }  
6 public class Main {  
7     public static void main(String[] args) {  
8         Demo d = null;  
9         d.method();  
10    }  
11 }
```

- a) In "Static" mà không lỗi
b) Lỗi NullPointerException
c) Lỗi compile
d) Không in gì cả

15. [Object Initialization - Basic] Biến nào dưới đây KHÔNG được Java khởi tạo tự động?

```
1 class Box {  
2     int a; // 1  
3     static int b; // 2  
4     String s; // 3  
5     boolean flag; // 4  
6 }
```

- a) Không có dòng nào
b) Chỉ dòng 1 (biến instance nguyên thủy)
c) Chỉ dòng 2
d) Chỉ dòng 3 (biến String)

16. [Application - Basic] Khi thiết kế một hệ thống quản lý người dùng (User, Admin, Guest) có chức năng login, đăng bài, xóa bài, theo hướng OOP, giải pháp nào sau đây giúp mã dễ bảo trì và mở rộng?

- a) Dùng một class duy nhất với nhiều if-else theo loại người dùng
b) Dùng các class riêng kế thừa từ 'User', mỗi class override hành vi riêng



- c) Gộp tất cả hành vi vào class 'Admin'
 - d) Dùng static method cho từng loại người dùng
17. **[Application - Basic]** Trong một game phiêu lưu, bạn có nhiều loại NPC (nhân vật không điều khiển được): Merchant, QuestGiver, Healer. Mỗi loại có hành vi tương tác khác nhau khi người chơi tiếp cận. Thiết kế nào hợp lý nhất để quản lý hành vi tương tác?
- a) Tạo 1 class NPC duy nhất và dùng switch-case để xử lý từng loại
 - b) Tạo một interface **Interactable** với phương thức **interact()**, và mỗi NPC cụ thể sẽ implement interface đó
 - c) Dùng static method **interact()** trong mỗi NPC
 - d) Dùng enum để xác định loại NPC và hành vi tương ứng
18. **[Encapsulation]** Access modifier nào dùng để ẩn dữ liệu trong Java?
- a) public
 - b) private
 - c) protected
 - d) default
19. **[Polymorphism:]** Method overloading là dạng đa hình nào?
- a) Runtime
 - b) Compile-time
 - c) Dynamic
 - d) Static
20. **[Abstraction]** Một abstract class có thể chứa phương thức nào sau đây?
- a) Chỉ phương thức trừu tượng
 - b) Chỉ phương thức cụ thể
 - c) Cả phương thức trừu tượng và cụ thể
 - d) Không chứa phương thức
21. **[Encapsulation]** Mục đích chính của getter và setter là gì?
- a) Tăng tốc độ chương trình
 - b) Kiểm soát truy cập dữ liệu
 - c) Kế thừa phương thức
 - d) Triển khai đa hình
22. **[Inheritance]** Từ khóa **super** dùng để làm gì?
- a) Gọi constructor của class con
 - b) Gọi constructor hoặc phương thức của class cha
 - c) Khai báo biến static
 - d) Triển khai interface
23. **[Polymorphism]** Khi class con ghi đè phương thức của class cha, đó là:
- a) Method overloading
 - b) Method overriding
 - c) Upcasting
 - d) Downcasting
24. **[Abstraction]** Một class có thể triển khai bao nhiêu interface trong Java?
- a) Chỉ 1
 - b) Chỉ 2
 - c) Không giới hạn
 - d) Không thể triển khai interface
25. **[Encapsulation]** Nếu một thuộc tính được khai báo **private**, nó có thể truy cập từ đâu?
- a) Mọi nơi
 - b) Trong cùng package
 - c) Chỉ trong cùng class
 - d) Trong class con
26. **[Polymorphism]** Upcasting là gì?
- a) Ép kiểu từ class cha xuống class con
 - b) Ép kiểu từ class con lên class cha
 - c) Ghi đè phương thức
 - d) Triển khai interface
27. **[Encapsulation]** Tại sao việc sử dụng **private** cho thuộc tính lại quan trọng?
- a) Tăng tốc độ chương trình
 - b) Đảm bảo dữ liệu được ẩn và kiểm soát
 - c) Hỗ trợ đa kế thừa
 - d) Triển khai phương thức trừu tượng
28. **[Inheritance]** Điều gì xảy ra nếu class con không gọi **super()** trong constructor?



- a) Chương trình tự động gọi constructor mặc định của class cha b) Báo lỗi runtime
c) Không tạo được object d) Class cha bị bỏ qua
29. **[Polymorphism]** Sự khác biệt chính giữa method overloading và method overriding là gì?
- a) Overloading dùng **super**, overriding dùng **this**
b) Overloading trong cùng class, overriding giữa cha-con
c) Overloading là runtime, overriding là compile-time
d) Overloading không cần kế thừa, overriding không cần tham số
30. **[Abstraction]** Tại sao abstract class không thể khởi tạo object trực tiếp?
- a) Vì nó không có constructor b) Vì nó chứa phương thức trừu tượng chưa triển khai
c) Vì nó là interface d) Vì nó chỉ dùng trong package
31. **[Encapsulation]** Làm thế nào để kiểm soát giá trị hợp lệ của một thuộc tính?
- a) Dùng getter để kiểm tra b) Dùng setter với logic kiểm tra
c) Dùng public cho thuộc tính d) Dùng interface
32. **[Inheritance]** Java có hỗ trợ đa kế thừa qua class không?
- a) Có, bằng từ khóa **extends** b) Không, dùng interface thay thế
c) Có, nhưng chỉ trong cùng package d) Không, dùng abstract class thay thế
33. **[Polymorphism]** Dynamic method dispatch hoạt động như thế nào?
- a) JVM gọi phương thức dựa trên kiểu tham chiếu b) JVM gọi phương thức dựa trên kiểu thực tế của object
c) Compile-time chọn phương thức d) Dùng **static** để gọi phương thức
34. **[Abstraction]** Khi nào nên dùng interface thay vì abstract class?
- a) Khi cần chia sẻ code cụ thể b) Khi cần định nghĩa giao diện chung và đa kế thừa
c) Khi cần biến instance d) Khi không cần phương thức trừu tượng
35. **[Encapsulation]** Lợi ích lớn nhất của việc ẩn thông tin (data hiding) là gì?
- a) Tăng hiệu suất chương trình b) Bảo vệ dữ liệu và giảm phụ thuộc
c) Hỗ trợ overriding d) Giảm kích thước mã nguồn
36. **[Polymorphism]** Upcasting và downcasting khác nhau như thế nào?
- a) Upcasting là thủ công, downcasting là tự động b) Upcasting từ con lên cha, downcasting từ cha xuống con
c) Upcasting dùng **super**, downcasting dùng **this** d) Upcasting là runtime, downcasting là compile-time
37. **[Encapsulation]** Kết quả của đoạn code sau là gì?

```
1 class Person {  
2     private String name = "John";  
3     public String getName() { return name; }  
4 }  
5  
6 public class Main {  
7     public static void main(String[] args) {  
8         Person p = new Person();  
9         System.out.println(p.getName());  
    }
```



```
10 }  
11 }
```

- a) John b) null c) Lỗi biên dịch d) Lỗi runtime

38. [Inheritance] Kết quả của đoạn code sau là gì?

```
1 class Animal {  
2     void sound() { System.out.println("Roar"); }  
3 }  
4 class Dog extends Animal {  
5 }  
6  
7 public class Main {  
8     public static void main(String[] args) {  
9         Dog d = new Dog();  
10        d.sound();  
11    }  
12 }
```

- a) Roar b) Woof
c) Không in gì d) Lỗi biên dịch

39. [Polymorphism] Kết quả của đoạn code sau là gì?

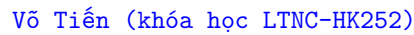
```
1 class Calc {  
2     int add(int a, int b) { return a + b; }  
3     double add(double a, double b) { return a + b; }  
4 }  
5  
6 public class Main {  
7     public static void main(String[] args) {  
8         Calc c = new Calc();  
9         System.out.println(c.add(2.5, 3.5));  
10    }  
11 }
```

- a) 5 b) 6.0
c) Lỗi biên dịch d) 6

40. [Abstraction] Kết quả của đoạn code sau là gì?

```
1 abstract class Shape {  
2     abstract int area();  
3 }  
4 class Rectangle extends Shape {  
5     int area() { return 10; }  
6 }  
7  
8 public class Main {  
9     public static void main(String[] args) {  
10        Rectangle r = new Rectangle();  
11        System.out.println(r.area());  
12    }  
13 }
```

- a) 0 b) 10
c) Lỗi biên dịch d) Lỗi runtime



```
1 class Account {
2     private int balance = 100;
3     public void setBalance(int b) { if (b > 0) balance = b; }
4     public int getBalance() { return balance; }
5 }
6
7 public class Main {
8     public static void main(String[] args) {
9         Account a = new Account();
10        a.setBalance(-50);
11        System.out.println(a.getBalance());
12    }
13 }
```

42. **[Inheritance]** Kết quả của đoạn code sau là gì?

```

1 class Parent {
2     Parent() { System.out.println("Parent"); }
3 }
4 class Child extends Parent {
5     Child() { System.out.println("Child"); }
6 }
7
8 public class Main {
9     public static void main(String[] args) {
10         Child c = new Child();
11     }
12 }

```

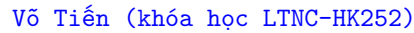
43. **[Polymorphism]** Kết quả của đoạn code sau là gì?

```

1  class Animal {
2      void move() { System.out.println("Walk"); }
3  }
4  class Fish extends Animal {
5      void move() { System.out.println("Swim"); }
6  }
7
8  public class Main {
9      public static void main(String[] args) {
10         Animal a = new Fish();
11         a.move();
12     }
13 }

```

44. **[Abstraction]** Kết quả của đoạn code sau là gì?



a) Car started b) Vehicle started
c) Lỗi biên dịch d) Không in gì

a) Alice b) null
c) Lỗi biên dịch d) Lỗi runtime

a) Flying high b) Flying low
c) Lỗi biên dịch d) Không in gì

Trang 9/16



- a) Dùng public cho tất cả thuộc tính b) Dùng private với getter/setter và logic kiểm tra trong setter
- c) Dùng abstract class d) Dùng interface
48. **[Abstraction]** Bạn muốn tạo một hệ thống với các loại xe (ô tô, xe máy) có hành vi chung là "di chuyển". Thiết kế nào hợp lý?
- a) Tạo từng class riêng lẻ không liên quan b) Tạo abstract class Vehicle với phương thức trừu tượng move()
- c) Dùng private cho tất cả phương thức d) Tạo class Vehicle với thuộc tính public
49. **[Inheritance]** Bạn cần một class con thừa hưởng các thuộc tính từ class cha như "động vật" (tên, tuổi) và thêm hành vi riêng như "sủa" cho chó. Thiết kế nào hợp lý?
- a) Dùng extends để kế thừa từ class cha b) Dùng interface cho tất cả hành vi
- c) Sao chép code từ class cha sang class con d) Dùng static cho các thuộc tính
50. **[Polymorphism]** Bạn muốn một phương thức "tính tổng" có thể hoạt động với cả số nguyên và số thực. Thiết kế nào hợp lý?
- a) Tạo hai phương thức riêng biệt với tên khác nhau b) Dùng method overloading với cùng tên add
- c) Dùng abstract class với phương thức trừu tượng d) Dùng interface để định nghĩa
51. **[Encapsulation]** Bạn cần đảm bảo dữ liệu như "số dư tài khoản" không bị thay đổi trực tiếp từ bên ngoài. Thiết kế nào hợp lý?
- a) Dùng public cho thuộc tính số dư b) Dùng private với getter và setter có kiểm tra
- c) Dùng protected để class con truy cập d) Không dùng getter/setter
52. **[Abstraction]** Bạn thiết kế một hệ thống với các loài chim, mỗi loài có cách bay khác nhau nhưng đều phải "bay". Thiết kế nào hợp lý?
- a) Dùng interface Flyable với phương thức fly() b) Dùng class thông thường với phương thức cụ thể
- c) Dùng static method cho hành vi bay d) Dùng nhiều class không liên kết
53. **[Inheritance + Abstraction]** Bạn muốn một class "Xe" vừa kế thừa từ "Phương tiện" vừa có khả năng "chở hàng" từ một giao diện khác. Thiết kế nào hợp lý?
- a) Dùng extends cho class cha và implements cho interface b) Dùng nhiều extends để kế thừa hai class
- c) Dùng private cho tất cả thuộc tính d) Dùng overloading cho các phương thức
54. **[Polymorphism]** Bạn cần một hệ thống mà hành vi "di chuyển" của động vật được quyết định bởi loại động vật cụ thể (chó chạy, chim bay). Thiết kế nào hợp lý?
- a) Dùng method overriding với class cha b) Dùng method overloading trong cùng class Animal
- c) Dùng static method cho mỗi loại d) Dùng interface mà không kế thừa
55. **[Encapsulation]** Bạn muốn hạn chế truy cập vào thuộc tính "mật khẩu" của class User nhưng vẫn cho phép thay đổi qua phương thức. Thiết kế nào hợp lý?
- a) Dùng public cho thuộc tính mật khẩu b) Dùng private với setter có kiểm tra
- c) Dùng abstract method để thay đổi mật khẩu d) Dùng protected để package truy cập
56. **[Abstraction]** Bạn thiết kế một hệ thống với các hình dạng (vuông, tròn) và cần tính diện tích riêng cho từng loại. Thiết kế nào hợp lý?
- a) Dùng abstract class Shape với phương thức trừu tượng area() b) Dùng interface không có phương thức cụ thể



- c) Tạo từng class riêng biệt không liên quan d) Dùng static method để tính diện tích
57. **[Abstraction + Inheritance]** Bạn thiết kế một game RPG với các nhân vật (Warrior, Mage, Archer) có chỉ số sức mạnh, mana, và kỹ năng riêng (như "chém", "phép thuật", "bắn tên"). Một số nhân vật đặc biệt (như Paladin) có thể vừa chém vừa dùng phép. Thiết kế nào hợp lý để quản lý kỹ năng?
- a) Dùng một abstract class Character với phương thức trừu tượng skill() và sao chép code cho từng kỹ năng
 - b) Dùng interface Attackable và Castable, kết hợp với extends Character để hỗ trợ đa dạng kỹ năng
 - c) Dùng method overloading trong class Character để xử lý mọi kỹ năng
 - d) Dùng static method trong mỗi class nhân vật để định nghĩa kỹ năng
58. **[Encapsulation + Abstraction]** Trong game bắn súng, bạn có các loại vũ khí (Pistol, Rifle, Sniper) với hành vi "bắn" khác nhau (tốc độ, sát thương). Vũ khí cũng có thể nâng cấp (tăng sát thương). Bạn cần đảm bảo chỉ class Weapon quản lý sát thương, không cho code bên ngoài thay đổi trực tiếp. Thiết kế nào hợp lý?
- a) Dùng public int damage trong class Weapon và để các class con tự tăng
 - b) Dùng private int damage với getter và phương thức upgrade() trong abstract class Weapon, các class con override shoot()
 - c) Dùng interface Upgradeable với public thuộc tính để nâng cấp
 - d) Dùng protected int damage để class con truy cập trực tiếp
59. **[Inheritance + Abstraction]** Trong game chiến thuật, bạn có các đơn vị lính (Infantry, Tank, Aircraft) với hành vi "di chuyển" và "tấn công" khác nhau. Một số đơn vị (như Helicopter) cần vừa di chuyển trên không vừa tấn công mặt đất. Bạn cũng muốn tái sử dụng code "tấn công" từ class cha. Thiết kế nào hợp lý?
- a) Dùng abstract class Unit với move() và attack(), kết hợp interface Flyable cho các đơn vị bay
 - b) Dùng nhiều class riêng biệt với static method cho từng hành vi
 - c) Dùng method overloading trong class Unit để xử lý mọi kiểu di chuyển và tấn công
 - d) Dùng một interface duy nhất UnitBehavior với tất cả hành vi
60. **[Polymorphism]** Bạn thiết kế game platformer với các kẻ thù (Zombie, Ghost, Boss) có hành vi "tấn công" khác nhau (cắn, bay qua, triệu hồi). Boss có thể thay đổi hành vi tấn công dựa trên máu (dưới 50% thì triệu hồi nhiều hơn). Thiết kế nào hợp lý để xử lý hành vi động này?
- a) Dùng abstract class Enemy với attack(), các class con override và dùng điều kiện trong attack()
 - b) Dùng interface Attackable với static method để thay đổi hành vi
 - c) Dùng method overloading trong class Boss để mô phỏng các kiểu tấn công
 - d) Dùng một class Enemy với tất cả hành vi trong switch-case
61. **[Encapsulation + Abstraction]** Trong game mô phỏng, bạn có các công trình (House, Factory, Tower) với thuộc tính "máu" và "năng lượng". Tower cần tự động tấn công khi kẻ thù đến gần, trong khi Factory sản xuất đơn vị. Bạn muốn ẩn dữ liệu "máu" và cho phép các công trình có hành vi đặc thù. Thiết kế nào hợp lý?
- a) Dùng public int health và interface Action với các phương thức tùy chỉnh
 - b) Dùng abstract class Structure với private int health và phương thức trừu tượng act(), các class con triển khai hành vi
 - c) Dùng protected int health và method overloading cho từng hành vi
 - d) Dùng một class Structure với tất cả hành vi trong điều kiện if-else
62. **[Encapsulation]** Trong hệ thống quản lý nhân viên, bạn cần bảo vệ thuộc tính "lương" khỏi thay đổi trực tiếp và chỉ cho phép tăng lương qua quy trình kiểm duyệt. Thiết kế nào hợp lý?



```
1 class Employee {
2     // Thiết kế A
3     public double salary;
4
5     // Thiết kế B
6     private double salary;
7     public double getSalary() { return salary; }
8     public void increaseSalary(double amount) {
9         if (amount > 0) salary += amount;
10    }
11 }
12
13 public class Main {
14     public static void main(String[] args) {
15         Employee emp = new Employee();
16         // A: emp.salary = -1000;
17         // B: emp.increaseSalary(500);
18         System.out.println(emp.getSalary());
19     }
20 }
```

- a) Thiết kế A: Dùng public để truy cập trực tiếp b) Thiết kế B: Dùng private với getter và setter có kiểm tra
c) Dùng abstract class để quản lý lương d) Dùng interface để định nghĩa lương
63. [Inheritance] Trong ứng dụng đặt hàng, bạn có các loại đơn hàng (OnlineOrder, InStoreOrder) với thông tin chung (ID, ngày đặt) và hành vi riêng (giao hàng, nhận tại cửa hàng). Thiết kế nào hợp lý?

```
1 // Thiết kế A
2 class Order {
3     String id;
4     String date;
5 }
6 class OnlineOrder extends Order {
7     void deliver() { System.out.println("Delivering"); }
8 }
9 class InStoreOrder extends Order {
10    void pickup() { System.out.println("Picking up"); }
11 }
12
13 // Thiết kế B
14 class OnlineOrder {
15     String id;
16     String date;
17     void deliver() { System.out.println("Delivering"); }
18 }
19 class InStoreOrder {
20     String id;
21     String date;
22     void pickup() { System.out.println("Picking up"); }
23 }
24
25 public class Main {
26     public static void main(String[] args) {
27         OnlineOrder o = new OnlineOrder();
28         o.deliver();
29     }
30 }
```



- a) Thiết kế A: Dùng extends để kế thừa từ Or- b) Thiết kế B: Tạo hai class riêng biệt không
der kế thừa
c) Dùng interface thay vì class cha d) Dùng static cho thông tin chung

64. **[Polymorphism]** Trong hệ thống thanh toán, bạn cần xử lý các phương thức thanh toán (CreditCard, PayPal) với hành vi "thanh toán" khác nhau. Thiết kế nào hợp lý?

```
1 // Thiết kế A
2 class Payment {
3     void pay() { System.out.println("Paid"); }
4 }
5 class CreditCard extends Payment {
6     void pay() { System.out.println("Paid by Credit Card"); }
7 }
8 class PayPal extends Payment {
9     void pay() { System.out.println("Paid by PayPal"); }
10 }
11
12 // Thiết kế B
13 class Payment {
14     void payCredit() { System.out.println("Paid by Credit Card"); }
15     void payPayPal() { System.out.println("Paid by PayPal"); }
16 }
17
18 public class Main {
19     public static void main(String[] args) {
20         Payment p = new CreditCard();
21         p.pay();
22     }
23 }
```

- a) Thiết kế A: Dùng overriding với class cha b) Thiết kế B: Dùng overloading trong cùng
Payment class Payment
c) Dùng interface thay vì class cha d) Dùng static method cho từng loại

65. **[Abstraction]** Trong ứng dụng quản lý tài liệu, bạn có các loại tài liệu (PDF, Word) với hành vi "mở" và "in" khác nhau. Thiết kế nào hợp lý?

```
1 // Thiết kế A
2 abstract class Document {
3     abstract void open();
4     abstract void print();
5 }
6 class PDF extends Document {
7     void open() { System.out.println("Open PDF"); }
8     void print() { System.out.println("Print PDF"); }
9 }
10 class Word extends Document {
11     void open() { System.out.println("Open Word"); }
12     void print() { System.out.println("Print Word"); }
13 }
14
15 // Thiết kế B
16 interface Document {
17     void open();
18     void print();
19 }
20 class PDF implements Document {
21     public void open() { System.out.println("Open PDF"); }
22     public void print() { System.out.println("Print PDF"); }
```



```
23 }
24
25 public class Main {
26     public static void main(String[] args) {
27         Document doc = new PDF();
28         doc.open();
29     }
30 }
```

- a) Thiết kế A: Dùng abstract class Document b) Thiết kế B: Dùng interface Document
c) Dùng class thông thường với thuộc tính public d) Dùng hai class riêng biệt không liên quan

66. [Polymorphism] Trong ứng dụng quản lý kho, bạn có các sản phẩm (Electronics, Clothing) với cách tính thuế khác nhau (10% cho điện tử, 5% cho quần áo). Thiết kế nào hợp lý?

```
1 // Thiết kế A
2 class Product {
3     double price;
4     Product(double price) { this.price = price; }
5     double calculateTax() { return 0; }
6 }
7 class Electronics extends Product {
8     Electronics(double price) { super(price); }
9     double calculateTax() { return price * 0.1; }
10 }
11 class Clothing extends Product {
12     Clothing(double price) { super(price); }
13     double calculateTax() { return price * 0.05; }
14 }
15
16 // Thiết kế B
17 class Product {
18     double price;
19     Product(double price) { this.price = price; }
20     double calculateTax(String type) {
21         if (type.equals("Electronics")) return price * 0.1;
22         return price * 0.05;
23     }
24 }
25
26 public class Main {
27     public static void main(String[] args) {
28         Product p = new Electronics(1000);
29         System.out.println(p.calculateTax());
30     }
31 }
```

- a) Thiết kế A: Dùng overriding với class cha b) Thiết kế B: Dùng overloading với tham số type
c) Dùng interface để tính thuế d) Dùng static method cho từng loại

67. Trong hệ thống giao dịch chứng khoán, bạn cần thiết kế các loại tài sản tài chính (Stock - cổ phiếu, Bond - trái phiếu, Option - quyền chọn) với các đặc điểm sau:

- Mỗi tài sản có thuộc tính cơ bản như "mã giao dịch" (code), "giá hiện tại" (price), và "khối lượng" (volume), nhưng cách tính "giá trị danh mục" (portfolio value) khác nhau:
 - Stock: Giá trị = Giá hiện tại × Khối lượng.
 - Bond: Giá trị = Giá hiện tại × Khối lượng + Lãi suất cố định (fixed interest).



- Option: Giá trị phụ thuộc vào "giá thực thi" (strike price) và điều kiện thị trường (giá trị = $\max(0, \text{Giá hiện tại} - \text{Giá thực thi}) \times \text{Khối lượng}$).
- Giá hiện tại cần được bảo vệ khỏi thay đổi trực tiếp, chỉ cập nhật qua phương thức kiểm soát (ví dụ: giá không âm).
- Một số tài sản (như Option) có thể "thực thi" (exercise), trong khi các loại khác không cần.
- Hệ thống phải dễ mở rộng cho các loại tài sản mới (như Futures) trong tương lai.

```
1 // Thiết kế A: Abstract Class + Interface
2 abstract class FinancialAsset {
3     private String code;
4     private double price;
5     private int volume;
6
7     FinancialAsset(String code, double price, int volume) {
8         this.code = code;
9         setPrice(price);
10        this.volume = volume;
11    }
12
13    public void setPrice(double price) {
14        if (price >= 0) this.price = price;
15    }
16    public double getPrice() { return price; }
17
18    abstract double calculatePortfolioValue();
19 }
20
21 interface Exercisable {
22     void exercise();
23 }
24
25 class Stock extends FinancialAsset {
26     Stock(String code, double price, int volume) { super(code, price, volume); }
27     double calculatePortfolioValue() { return getPrice() * volume; }
28 }
29
30 class Bond extends FinancialAsset {
31     private double interest;
32     Bond(String code, double price, int volume, double interest) {
33         super(code, price, volume);
34         this.interest = interest;
35     }
36     double calculatePortfolioValue() { return getPrice() * volume + interest; }
37 }
38
39 class Option extends FinancialAsset implements Exercisable {
40     private double strikePrice;
41     Option(String code, double price, int volume, double strikePrice) {
42         super(code, price, volume);
43         this.strikePrice = strikePrice;
44     }
45     double calculatePortfolioValue() {
46         return Math.max(0, getPrice() - strikePrice) * volume;
47     }
48     public void exercise() { System.out.println("Option exercised"); }
49 }
```



```
1 // Thiết kế B: Một Class với Switch-case
2 class FinancialAsset {
3     private String code;
4     private double price;
5     private int volume;
6     private String type; // "Stock", "Bond", "Option"
7     private double interest; // Cho Bond
8     private double strikePrice; // Cho Option
9
10    FinancialAsset(String code, double price, int volume, String type, double interest, double strikePrice) {
11        this.code = code;
12        this.price = price;
13        this.volume = volume;
14        this.type = type;
15        this.interest = interest;
16        this.strikePrice = strikePrice;
17    }
18
19    double calculatePortfolioValue() {
20        switch (type) {
21            case "Stock": return price * volume;
22            case "Bond": return price * volume + interest;
23            case "Option": return Math.max(0, price - strikePrice) * volume;
24            default: return 0;
25        }
26    }
27
28    void exercise() {
29        if (type.equals("Option")) System.out.println("Option exercised");
30    }
31 }
```

- a) Thiết kế A: Dùng abstract class FinancialAsset và interface Exercisable
- b) Thiết kế B: Dùng một class FinancialAsset với switch-case xử lý loại tài sản
- c) Dùng interface duy nhất FinancialAsset với tất cả phương thức trừu tượng
- d) Dùng class cha FinancialAsset với public thuộc tính và overloading cho calculatePortfolioValue